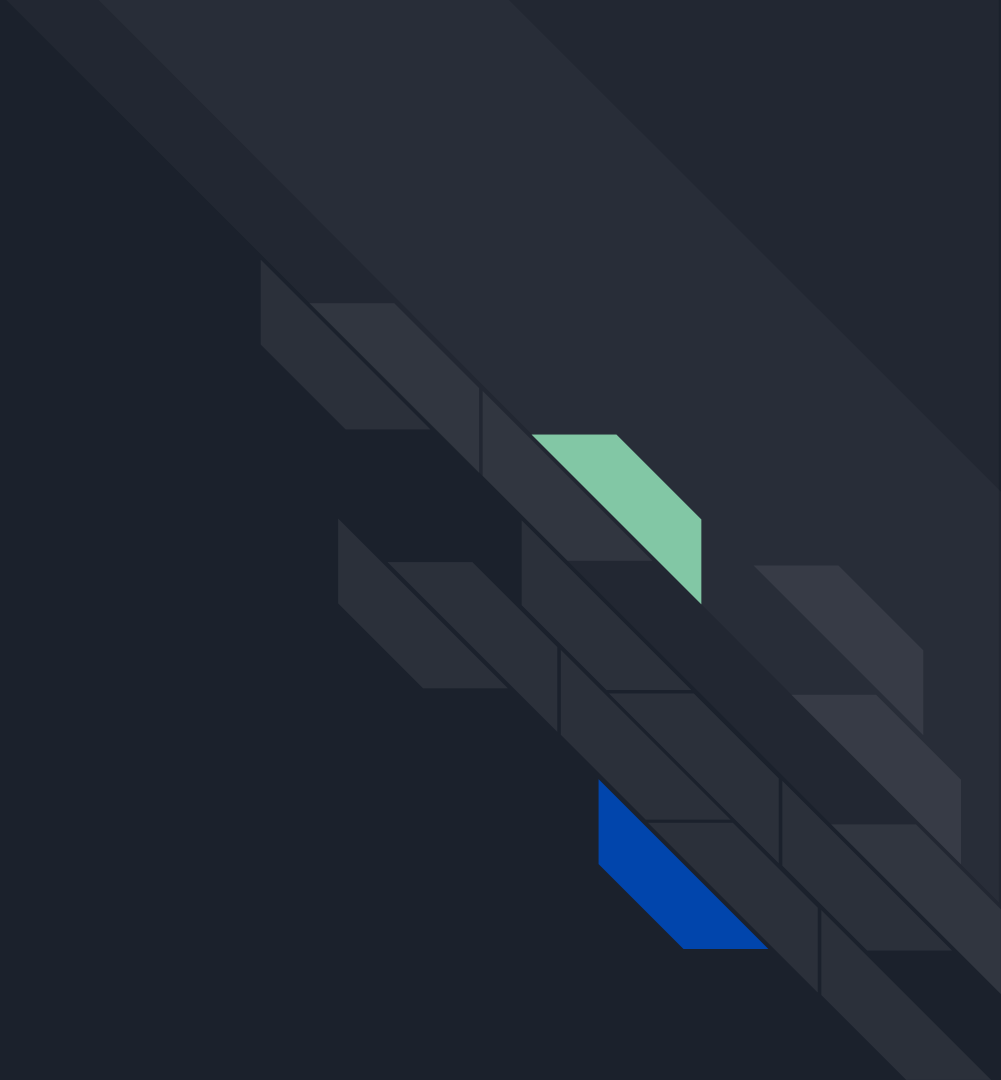


A decorative graphic on the left side of the slide consisting of two overlapping parallelograms. The front one is blue and the back one is a light green. They are positioned diagonally, with the blue one partially covering the green one.

Emotion Analysis in Natural Language Processing

Team 29

Introduction





Topic

When we communicate through text, words alone are sometimes insufficient to accurately convey our emotions, which can lead to misunderstandings of others' sentiments.

By combining machine learning and deep learning techniques, we train AI models to better recognize and interpret the emotions that others intend to express.

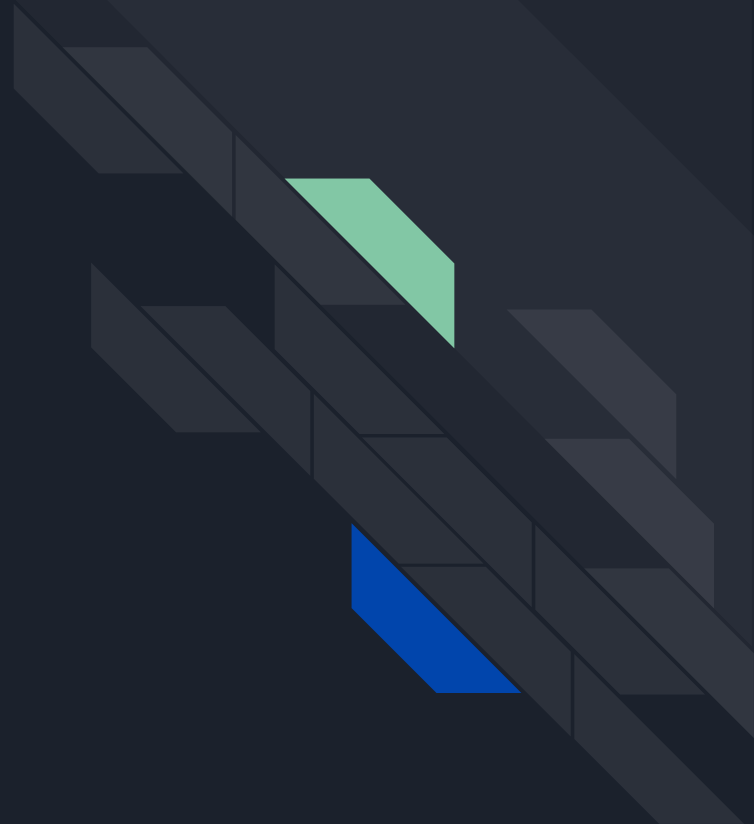


Progress and goals

We will design an APP, user can input a sentence, then it will output original sentence with emoji.

For example, when user input “I love you”, it will output “I love you❤️”. By automatic adding emoji, making sentence sentiment more understandable.

Related Work





Previous work

Text Sentiment Analysis System: Sentiment Polarity Detection and Analysis Based on Tweet Content

<https://github.com/fightingantony/2024-AI-final-project>

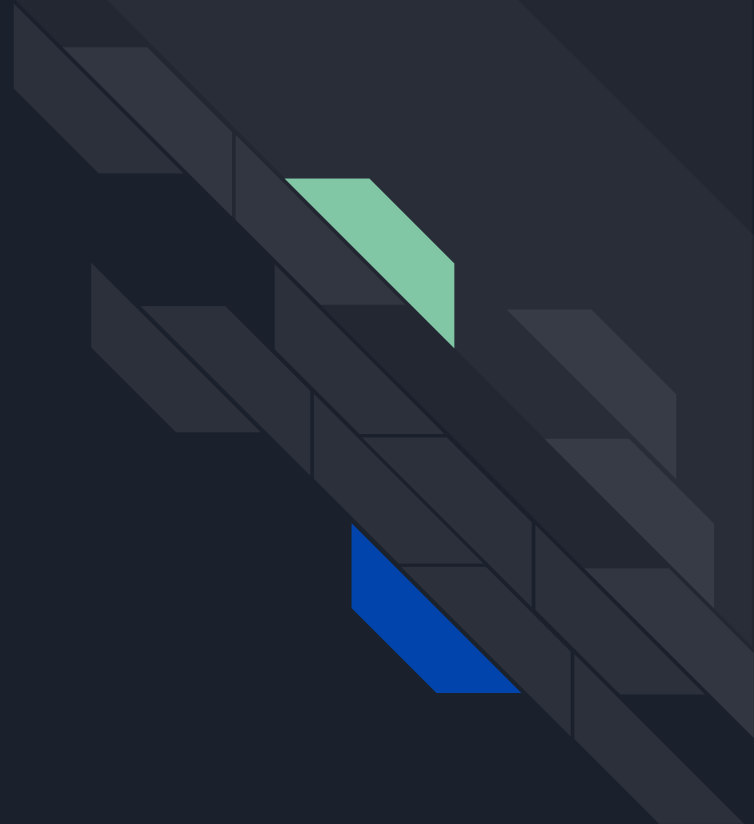
comparison

we will output addition emoji to clearly show the emotion of the sentence.

And analysing accurater emotion than Sentiment.

We used MLP instead of stacking ensemble model with logistic regression and random forest.

Dataset





Dataset

[cirimus/super-emotion](#) · [Datasets at Hugging Face](#)

there are a total of 552.k data in the dataset, each data contain a sentence and correspond emotions.

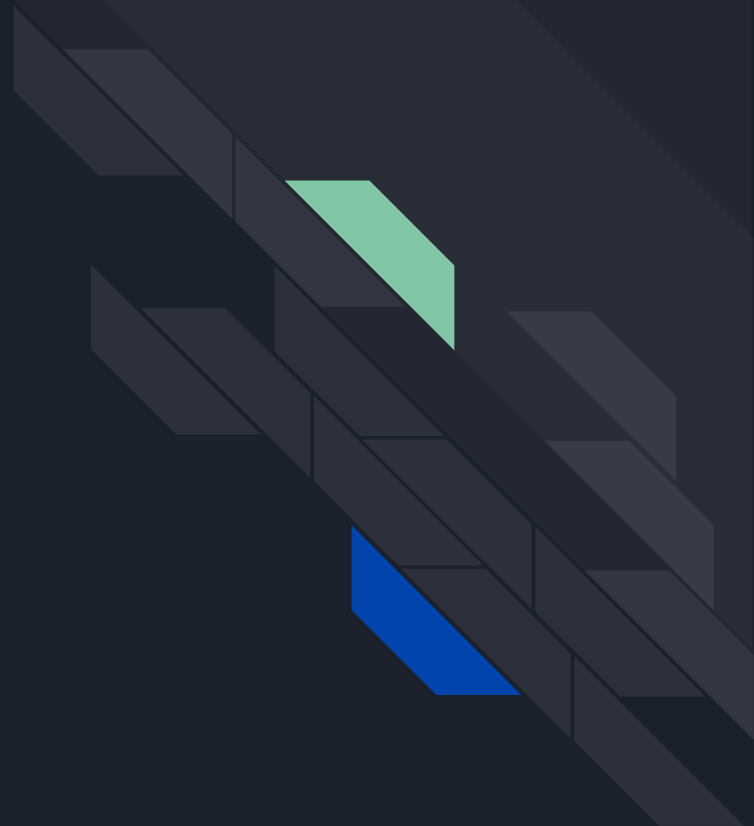
covering 7 classes of emotion: anger, fear, joy, love, neutral, sadness, surprise

some sentences may contain multiple emotions, in this project, **we only take the first in labels.**

distribution: anger(0) - 75477, fear(1) - 6015, joy(2) - 166753, love(3) - 56490,

neutral(4) - 31683, sadness(5) - 137894, surprise(6) - 24373

Baseline





Baseline

Term Frequency-Inverse Document Frequency(TD-IDF):

a statistical measure used to evaluate how important a word is to a document in a collection (corpus). It's widely used in search engines, text mining, and information retrieval.

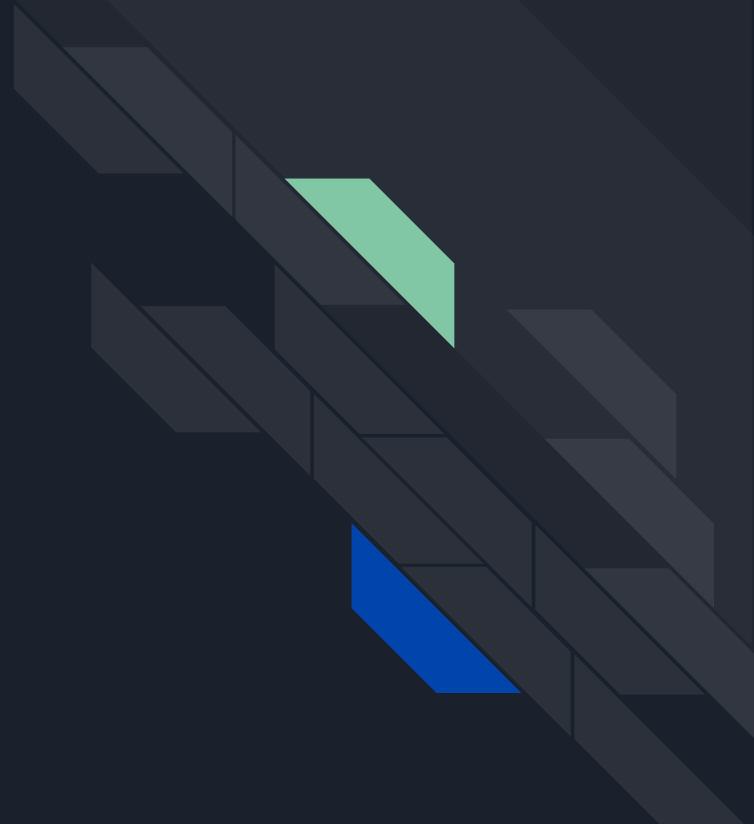
Model:

In this project, For text classification, we choose Multi-Layer Perceptron(MLP) as baseline.

It contain 3 Linear layers, 2 ReLU activation layers and 2 Dropout layers

The struct: Linear $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ Linear $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ Linear

Main Approach





1.Input and Preprocessing

This step ensure the text data clean, standarlized, and ready for feature extraction:

Input:

- Raw English text

Preprocessing steps:

- Convert text to lowercase.
- Tokenize the text (using `nltk.word_tokenize`).
- Remove punctuation, numbers, and stopwords.

Batch Processing:

- Perform in mini-batches to improve memory efficiency and processing speed.
- The cleaned text is saved in batch-wise JSONL format for efficient loading and subsequent TF-IDF vectorization.



2.feature extract

Term Frequency - Inverse Document Frequency(TF-IDF) vectorization

Implement a batch-wise TF-IDF vectorization process to efficiently convert text data into feature tensors. Each mini-batch consists of 50 samples, with each sample represented as a fixed-length TF-IDF vector.

Batch processing also helps reduce memory overhead and enables parallel computation on GPU.

Then, we get TF-IDF vectors, which represent the importance of each word in the context of the document and the entire corpus. This allows our MLP model to learn meaningful patterns for emotion classification.



3.Single-Label Conversion and Data Preparation

Simplifying a multi-label emotion classification problem into a single-label task by selecting only the first label from each sample. Then, preparing the training and testing data by pairing TF-IDF feature batches with the corresponding single-label targets, converting them into tensors for model training.

```
y_train_single = [labels[0] for labels in labels_train]
y_test_single = [labels[0] for labels in labels_test]
#轉成tensor
train_data = []
start_idx = 0
for batch in x_train_batches:
    current_batch_size = batch.shape[0]
    end_idx = start_idx + current_batch_size
    train_data.append((batch, torch.tensor(y_train_single[start_idx:end_idx], dtype=torch.long)))
    start_idx = end_idx
test_data = []
start_idx = 0
for batch in x_test_batches:
    current_batch_size = batch.shape[0]
    end_idx = start_idx + current_batch_size
    test_data.append((batch, torch.tensor(y_test_single[start_idx:end_idx], dtype=torch.long)))
    start_idx = end_idx
```



4. Model Training

We use a 3-layer multilayer perceptron. Between each layer, there is a ReLU activation layer and a dropout layer.

The ReLU layer enables the model to learn nonlinear relationships.

The dropout layer helps prevent overfitting.

```
#建立模型
class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super(MLP, self).__init__()
        self.net=nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(hidden_dim, hidden_dim//2),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(hidden_dim//2, output_dim)
        )
    def forward(self, x):
        return self.net(x)
```



4. Model Training

The dataset is imbalanced, so we adjust the class weights accordingly.

#設定類別的權重，因為資料集不平衡

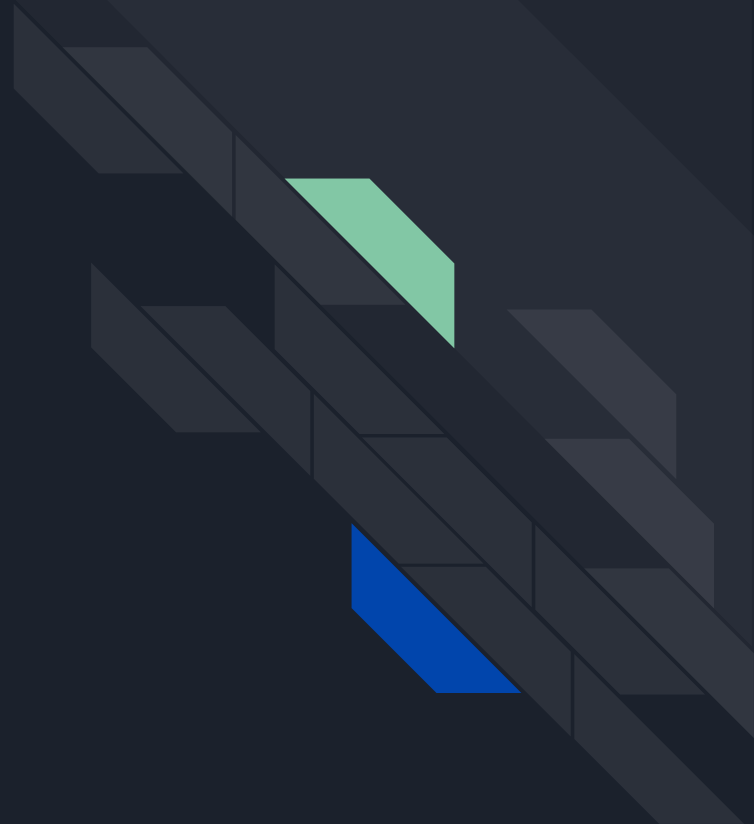
```
class_weights = compute_class_weight(  
    class_weight='balanced',  
    classes=np.arange(7),  
    y=y_train_single  
)  
class_weights = torch.tensor(class_weights, dtype=torch.float32)
```

Train the model for 50 epochs using batch training.

For each batch, compute weighted loss to handle class imbalance, update model parameters via backpropagation, and record loss and accuracy.

After each epoch, print average loss and accuracy to monitor training progress.

Evaluation Metric





Evaluation Metric

- Accuracy: Number of correct predict testcases/number of all testcases, the proportion of correct predictions over all predictions.
- Precision : When model predicts a certain class, how often is it correct?
- Recall : Of all the actual instances of a class, how many did the model correctly identify?
- F1-score : $2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$

Accuracy	Precision	Recall	F1-score
76.7%	69.4%	71.4%	70%



Evaluation Metric

Confusion matrix: row:ground true class, column:predict class

```
[[ 6654   391   212    48   602   454   258]
 [   300  4493   241    35   283   274   496]
 [   267   160 14086  1351   673   252   465]
 [   228    97  1040  3745   547   167   192]
 [   415   156   358    46  2160   153   507]
 [   631   467   313    80   315 12165   189]
 [   118   190   219    38   357    69 1668]]
```



Evaluation Metric

Classification Report: Showing scores for each evaluation metric.

Classification Report:					
	precision	recall	f1-score	support	
0	0.77	0.77	0.77	8619	
1	0.75	0.73	0.74	6122	
2	0.86	0.82	0.84	17254	
3	0.70	0.62	0.66	6016	
4	0.44	0.57	0.49	3795	
5	0.90	0.86	0.88	14160	
6	0.44	0.63	0.52	2659	
accuracy			0.77	58625	
macro avg	0.69	0.71	0.70	58625	
weighted avg	0.78	0.77	0.77	58625	

Results & analysis &
Others





Type of experiment

We try many way to improve accuracy of model, but unfortunately, we did not keep detailed records of the results for each change. We only provide summary in words.

- Changing epoch: Changing epoch can find the best times to training model, we found that if epoch if more than 75, then overfitting will happen. Furthermore, in fact, epoch have nearly performance when it is 30 and 50. So we think changing epoch not make great change.
- Batching: At first, the dataset is too large to be processed, we only use 1/4 data to training model the accuracy is 68%. Later we use batch to split data in batch size=1000, but we can't get real data about either batching data is good or bad for training model. Because we didn't find batching make a bug, we found it after we had changed model, so accuracy change may be result from many reason.



Type of experiment

- BatchNorm1d: We tried BatchNorm1d in model. But the accuracy decrease because of overfitting, and accuracy still smaller when modify epoch. The following is reason we guess:
 - Batch size is too small, so match normalization perform bad.
 - In TF/IDF vectorization the data is already normalized and very sparse. Using Batch Normalization may change the original meaning of the features, making the model harder to learn.

Discussion and analysis

In confusion matrix and classification report, there are some feature :

1. Joy and love are often confused with each other.
2. Neutral is very often misclassified as other sentiment and hard to correct recognize.

	anger	fear	joy	love	neutral	sad	surprise
[[	6654	391	212	48	602	454	258]
[	300	4493	241	35	283	274	496]
[	267	160	14086	1351	673	252	465]
[	228	97	1040	3745	547	167	192]
[	415	156	358	46	2160	153	507]
[	631	467	313	80	315	12165	189]
[	118	190	219	38	357	69	1668]]]

Classification Report:

	precision	recall	f1-score	support
0	0.77	0.77	0.77	8619
1	0.75	0.73	0.74	6122
2	0.86	0.82	0.84	17254
3	0.70	0.62	0.66	6016
4	0.44	0.57	0.49	3795
5	0.90	0.86	0.88	14160
6	0.44	0.63	0.52	2659



Discussion and analysis

1. The reason for the confusion between love and joy, we guess, is that both sentiments are positive, so their word representations in vectorization are similar.
2. Neutral sentences often lack clear sentiment words like "happy" or "hate." This causes the model to frequently misclassify them. For example, if a neutral sentence contains some strong sentiment words, it might be wrongly predicted as another sentiment, which lowers the recall for neutral. Conversely, sentences with mild sentiment may be predicted as neutral, lowering precision for the neutral class.

Limitation of work

- TF-IDF does not capture word order or contextual meaning.

For example, sentences like “I am not happy” and “I am happy” may result in nearly identical TF-IDF representations, since TF-IDF ignores word order and the presence of negations, which are critical for emotion classification.

- The model cannot handle out-of-vocabulary words effectively.
- The MLP is a shallow model with limited capacity to capture complex emotion patterns.
- The model only classify one sentiment.

Result

I am not happy 😊

Result

I am happy 😊

wrong classification



Practical Use

Translate: In translation, some emotions in sentences may not be conveyed by pure text translation

reading assistant: Some people may be unable to perceive emotions in text properly due to illness or other factors. The model can Label the emotions for reader

In our project, We created a simple website that can analyze the emotions of different sentences and add corresponding emojis.



Github Link and Reference

Github Link: [wayne777aa/AI_Final_Project](#)

Reference:

Previous Work: [fightingantony/2024-AI-final-project: Text Sentiment Analysis System: Sentiment Polarity Detection and Analysis Based on Tweet Content](#)

Dataset: [cirimus/super-emotion · Datasets at Hugging Face](#)

NLTK: [NLTK :: Natural Language Toolkit](#)

MLP-Pytorch: [MLP — Torchvision main documentation](#)

sklearn: [https://scikit-learn.org/stable/](#)



Contribution of each member

陳楷元:40%, model building, model training, slides, dataset finding.

蔡懷恩:30%, data input, feature extraction

許登壹:30%, model test, web building, slides.